

Step 4 — A Router That Sends Each Message to the Right Agent Automatically — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — Step 4: after Step 3, you choose the agent by typing `/extract`. This step hands that choice to a router, gives the Concierge its own file, and moves the "router → agent" flow into a service. That turns two modes of one script into a real 2-agent system. **If not:** users would have to know and type a command to file a ticket — exactly the manual sorting this project exists to remove.

After this step the project is complete. Files from Steps 1-3 that aren't shown here (`config.py` , `exceptions.py` , `models/llm.py` , `agents/triage.py` , `prompts/concierge.txt` , `prompts/triage.txt` , everything in `utils/` , `tests/unit/test_history.py` , `scripts/triage_samples.py`) don't change.

Basic Version

Approach 1 — a keyword router

Story — `agents/router.py` (Basic): a list of complaint words; if any appears, it's Triage. Free and instant, no API call. **If not:** there'd be nothing to replace `/extract`.

```
# src/supportdesk/agents/router.py
TRIGGER_WORDS = ["charged", "broken", "refund", "crash", "can't log"]

def decide(message: str) -> str:
    lowered = message.lower()
    for word in TRIGGER_WORDS:
        if word in lowered:
            return "triage"
    return "concierge"
```

Story — `agents/concierge.py` (Basic): Step 3's chat branch, moved into its own file so it's an agent like Triage. **If not:** one of the "two agents" would still just be a branch inside `main.py`.

```
# src/supportdesk/agents/concierge.py
from openai import OpenAI
```

```

from supportdesk.models.llm import stream_message
from supportdesk.utils.history import trim_history
from supportdesk.utils.prompts import load_prompt

MAX_HISTORY_TOKENS = 3000

def start_history() -> list[dict]:
    return [{"role": "system", "content": load_prompt("concierge")}]

def concierge_agent(client: OpenAI, history: list[dict], message: str) -> str:
    trim_history(history, MAX_HISTORY_TOKENS)
    return stream_message(client, history, message)

```

Story — `main.py` (Basic): the loop asks the router, then calls the agent it picked. **If not:** the router would exist but nothing would use it.

```

# src/supportdesk/main.py
import sys

from supportdesk.agents.concierge import concierge_agent, start_history
from supportdesk.agents.router import decide
from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import check_api_key_at_startup, create_client

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    if not check_api_key_at_startup(client):
        print("Login failed – check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = start_history()
    while True:
        message = input("You: ").strip()
        if message == "/quit":

```

```
        break
    if decide(message) == "triage":
        print(triage_agent(client, message))
    else:
        print("Concierge: ", end="", flush=True)
        concierge_agent(client, history, message)

if __name__ == "__main__":
    main()
```

This works for clearly worded messages. But nothing is logged, so a wrong route can't be explained. A complaint without a trigger word ("the app freezes on login") goes to the Concierge, and "what's your refund policy?" becomes a ticket just because it says "refund". This version also dropped Step 3's error handling.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a model-call router with a logged reason, and a service layer

Changed from Step 3: `SupportTicket` is unchanged; `RouteDecision` is added below it.

Story — `schemas.py`: adds `RouteDecision` next to `SupportTicket`. The router's answer is structured output too, the way Doc04's table suggests for "a supervisor picks the next agent". **If not:** you'd be parsing a free-text reply like "I think triage." by hand, and a typo'd agent name could slip through.

```
# src/supportdesk/schemas.py
from typing import Literal

from pydantic import BaseModel

class SupportTicket(BaseModel):
    # why: str | None, not str – a required name would force the model
    # to invent one when the customer never gave it.
    customer_name: str | None
    # how: Literal means the model can only pick one of these exact values
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str

# new in Step 4
class RouteDecision(BaseModel):
    # why: agent names as a Literal – a typo'd agent name can't come back
    agent: Literal["concierge", "triage"]
    # why: the reason goes into the log, so a wrong route can be explained
    reason: str
```

Story — `prompts/router.txt`: says exactly what counts as "triage" and what counts as "concierge", and asks for a short reason. **If not:** the model would guess its own dividing line, and it would move from message to message.

`src/supportdesk/prompts/router.txt`

You route messages for a customer support desk. Choose 'triage' if the message reports a problem that needs a support ticket (a wrong charge, something broken, no access to an account). Choose 'concierge' for greetings, small talk and general questions. Give a one-sentence reason.

Story — `agents/router.py` : one small structured call per message, which understands wording no keyword list covers. It starts from a safe default (Concierge) and only replaces it with a usable answer. Every decision is logged with its reason. **If not:** unclear messages would be misrouted with no trace, and a failed routing call could crash the loop or file a false ticket.

```
# src/supportdesk/agents/router.py
from openai import OpenAI
from pydantic import ValidationError

from supportdesk.models.llm import MODEL
from supportdesk.schemas import RouteDecision
from supportdesk.utils.logger import get_logger
from supportdesk.utils.prompts import load_prompt

def decide(client: OpenAI, message: str) -> RouteDecision:
    # when: logger fetched here, after load_config() has loaded .env
    logger = get_logger(__name__)
    messages = [
        {"role": "system", "content": load_prompt("router")},
        {"role": "user", "content": message},
    ]
    # why: the safe default – Concierge only chats, so if the router's
    # answer can't be used, no wrong ticket is ever created
    decision = RouteDecision(
        agent="concierge", reason="router could not decide"
    )
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=RouteDecision
        )
        # how: parsed is None when the model refused – keep the default
        reply = completion.choices[0].message
        if reply.parsed is not None:
            decision = reply.parsed
    except ValidationError as e:
        logger.warning("Router output did not match RouteDecision: %s", e)

    # why: every decision is logged with its reason, so a misrouted
    # message can be found and explained later
```

```

logger.info(
    "routed to %s (%s): %r", decision.agent, decision.reason, message
)
return decision

```

Moved from Step 3's `main.py` : the system-prompt line, `MAX_HISTORY_TOKENS` , and the trim + stream lines of the chat branch. Nothing new is added — they're just given their own file.

Story — `agents/concierge.py` : Step 3's chat branch becomes an agent: its own prompt (`start_history()`), its own token budget, one function to call. **If not:** the "Concierge" would only be a name for some lines inside `main.py` .

```

# src/supportdesk/agents/concierge.py
from openai import OpenAI

from supportdesk.models.llm import stream_message
from supportdesk.utils.history import trim_history
from supportdesk.utils.prompts import load_prompt

# why: far below gpt-4o-mini's real limit (128,000 tokens), so the rough
# 4-characters-per-token guess can be off and still be safe
MAX_HISTORY_TOKENS = 3000

def start_history() -> list[dict]:
    # why: the Concierge's own system prompt is what makes it a separate
    # agent from Triage, not one function with an if-statement.
    # when: called once, when the app starts – one conversation per run.
    system_prompt = load_prompt("concierge")
    return [{"role": "system", "content": system_prompt}]

def concierge_agent(client: OpenAI, history: list[dict], message: str) -> str:
    # why: trim BEFORE sending, so a long chat never hits the limit
    trim_history(history, MAX_HISTORY_TOKENS)
    return stream_message(client, history, message)

```

Moved + new: `print_ticket()` moves here unchanged from Step 3's `main.py` .

`handle_message()` is new — it holds the old `/extract -or-chat if` , but the router makes the choice now.

Story — `services/desk_service.py` : the whole flow in one function: ask the router, run the agent it picked, show the result. `print_ticket()` moves here from `main.py` . (Also create an empty `src/supportdesk/services/__init__.py` .) **If not:** `main.py` would mix keyboard input with business logic, and Project 5's web API would have to copy this flow instead of calling it.

```
# src/supportdesk/services/desk_service.py
from openai import OpenAI

from supportdesk.agents.concierge import concierge_agent
from supportdesk.agents.router import decide
from supportdesk.agents.triage import triage_agent
from supportdesk.schemas import SupportTicket
from supportdesk.utils.logger import get_logger

def print_ticket(ticket: SupportTicket) -> None:
    name = ticket.customer_name
    if name is None:
        # why: show clearly that no name was given – never a made-up one
        name = "(not given)"
    print("Ticket created:")
    print(f"  customer: {name}")
    print(f"  category: {ticket.issue_category}")
    print(f"  urgency: {ticket.urgency}")
    print(f"  summary: {ticket.summary}")

# new in Step 4
def handle_message(client: OpenAI, history: list[dict], message: str) -> None:
    # why: the whole "router picks an agent" flow lives here, not in
    # main.py – main.py only reads input, so a web API could later call
    # this same function without any terminal code.
    logger = get_logger(__name__)
    decision = decide(client, message)

    if decision.agent == "triage":
        ticket = triage_agent(client, message)
        if ticket is None:
            print("Sorry, I couldn't turn that into a ticket – please")
            print("describe the problem again in a little more detail.")
        else:
            logger.info(
                "ticket: %s / %s", ticket.issue_category, ticket.urgency
            )
            print_ticket(ticket)
    else:
        print("Concierge: ", end="", flush=True)
        concierge_agent(client, history, message)
```

Changed from Step 3: the startup and every `except` block are the same. What's gone: `print_ticket()`, `MAX_HISTORY_TOKENS` and the `if/else` body (all moved). What's new:

`history = start_history()` and one call, `handle_message(...)`, inside the `try`.

Story — `main.py` : now only startup, the input loop, and Step 3's error handling (unchanged).

`/extract` is gone. **If not:** `main.py` would keep growing with every new agent, instead of staying a thin entry point.

```
# src/supportdesk/main.py
import sys

import openai

from supportdesk.agents.concierge import start_history
from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import check_api_key_at_startup, create_client
from supportdesk.services.desk_service import handle_message
from supportdesk.utils.logger import get_logger

def main() -> None:
    # why: config first – a missing OPENAI_API_KEY stops the app here,
    # with one clear line, before anything else runs
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed – check OPENAI_API_KEY.")
        print("Login failed – check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = start_history()
    print("SupportDesk AI – type a message, or /quit to exit.")

    while True:
        message = input("You: ").strip()
        if message == "/quit":
            break
        if message == "":
            continue

        # changed in Step 4: one call replaces the /extract-or-chat if/else
        try:
```

```

    handle_message(client, history, message)
except openai.AuthenticationError:
    # how: a bad key is permanent – no retry fixes it, so exit
    logger.error("API key stopped working mid-session.")
    print("Login failed – check OPENAI_API_KEY in your .env file.")
    sys.exit(1)
except openai.RateLimitError:
    # how: the client already retried twice before this reached us
    logger.warning("Rate limited after the client's retries.")
    print("We're busy right now – wait a moment and try again.")
except openai.BadRequestError as e:
    logger.warning("Request rejected (likely too long): %s", e)
    # why: the message that broke the limit is still at the end of
    # history – remove it, or every next turn resends it and fails
    if history[-1]["role"] == "user":
        history.pop()
    print("That message was too long to handle – please shorten it.")

if __name__ == "__main__":
    main()

```

Changed from Step 3: the "trriage" list is the same; the "routing" list is new.

Story — data/sample/messages.json : adds routing cases — each message with the agent it should reach. **If not:** routing could only be checked by chatting by hand.

data/sample/messages.json

```

{
  "trriage": [
    "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I need it fixed today.",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush"
  ],
  "routing": [
    {"message": "hi there, how are you today?", "expected": "concierge"},
    {"message": "what are your support hours?", "expected": "concierge"},
    {
      "message": "I was charged twice this month, please fix it",
      "expected": "trriage"
    },
    {
      "message": "the app crashes every time I open settings",
      "expected": "trriage"
    }
  ]
}

```

```
]
}
```

(The first message is wrapped onto 2 lines here only to fit the page — in your file keep it on **one** line, because a JSON string can't contain a line break.)

Changed from Step 3: the import adds `RouteDecision` ; cases 1-2 are the same; `test_route_rejects_unknown_agent()` is new.

Story — `tests/unit/test_schemas.py` : one more case — `RouteDecision` rejects an agent name that doesn't exist. **If not:** someone could widen the `Literal` by mistake, and the router could return an agent `handle_message()` doesn't know.

```
# tests/unit/test_schemas.py
# Checks the Pydantic schemas accept good data and reject bad data.
# No API calls, no API key needed.
# Run from the repo root: python tests/unit/test_schemas.py
from pydantic import ValidationError

from supportdesk.schemas import RouteDecision, SupportTicket

def test_ticket_without_a_name() -> None:
    # a missing name must be allowed – the model should never invent one
    ticket = SupportTicket(
        customer_name=None,
        issue_category="account_access",
        urgency="medium",
        summary="Customer can't log in after two password resets.",
    )
    print(f"case 1 (no name allowed)      -> {ticket.customer_name}")

def test_ticket_rejects_unknown_urgency() -> None:
    try:
        SupportTicket(
            customer_name="Sarah Khan",
            issue_category="billing",
            urgency="urgent",
            summary="Charged twice.",
        )
    except ValidationError:
        print("case 2 (bad urgency)      -> ValidationError")

# new in Step 4
```

```
def test_route_rejects_unknown_agent() -> None:
    try:
        RouteDecision(agent="billing", reason="looks like billing")
    except ValidationError:
        print("case 3 (bad agent name)      -> ValidationError")

if __name__ == "__main__":
    test_ticket_without_a_name()
    test_ticket_rejects_unknown_urgency()
    test_route_rejects_unknown_agent()
```

Changed from Step 3: one new import (`decide`), one new function (`test_routing()`), and one new call at the bottom. Cases 1-4 are the same.

Story — `tests/integration/test_live_agents.py` : Step 3's checks plus case 5 — every routing case must reach its expected agent. **If not:** a small change to `router.txt` could quietly send complaints to the Concierge.

```
# tests/integration/test_live_agents.py
# Runs the project's checks against the real OpenAI API.
# Run from the repo root: python tests/integration/test_live_agents.py
# Makes real API calls – needs a valid OPENAI_API_KEY in .env.
import json

from openai import OpenAI

from supportdesk.agents.router import decide
from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)

SAMPLES_PATH = "data/sample/messages.json"

def load_samples() -> dict:
    with open(SAMPLES_PATH) as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def test_memory(client: OpenAI) -> None:
```

```

# case 1: turn 3 must be able to use what was said in turn 1
history = [{"role": "system", "content": "You are a helpful assistant."}]
send_message(client, history, "My order number is 4471.")
send_message(client, history, "Thanks, that's all for now.")
reply = send_message(client, history, "What was my order number?")
print(f"case 1 (memory)          -> {reply}")

def test_bad_key() -> None:
    # case 2: a wrong key is caught at startup, not mid-chat
    bad_client = OpenAI(api_key="sk-fake-key-123")
    key_ok = check_api_key_at_startup(bad_client)
    print(f"case 2 (bad key ok?)   -> {key_ok}")

def test_triage(client: OpenAI, samples: dict) -> None:
    # case 3: each sample complaint becomes a ticket
    for message in samples["triage"]:
        ticket = triage_agent(client, message)
        print(f"case 3 (triage)      -> {ticket}")

    # case 4: the second complaint gives no name – none may be invented
    second = triage_agent(client, samples["triage"][1])
    if second is not None and second.customer_name is None:
        print("case 4 (no name)      -> OK, customer_name is None")
    else:
        print("case 4 (no name)      -> WRONG, a name was invented")

# new in Step 4
def test_routing(client: OpenAI, samples: dict) -> None:
    # case 5: clear messages must reach the expected agent
    for case in samples["routing"]:
        decision = decide(client, case["message"])
        result = "OK"
        if decision.agent != case["expected"]:
            result = "WRONG"
        print(f"case 5 (routing)          -> {result}: {decision.agent}")

if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    samples = load_samples()
    test_memory(client)
    test_bad_key()

```

```
test_triage(client, samples)
test_routing(client, samples)
```

Run everything (from the repo root):

```
python tests/unit/test_history.py
python tests/unit/test_schemas.py
python tests/integration/test_live_agents.py
python -m supportdesk.main
```

Expected output of the new unit test line: `case 3 (bad agent name) -> ValidationError`. The integration test ends with four `case 5 (routing) -> OK: ... lines`.

Expected app session (the log lines go to the console too; the model's wording varies):

```
SupportDesk AI – type a message, or /quit to exit.
You: hi! what are your support hours?
... supportdesk.agents.router INFO routed to concierge (The user asks
a general question about support hours.): 'hi! what are your support
hours?'
Concierge: We're available Monday to Friday, 9am to 6pm.
You: can't log into my account, tried resetting password twice
... supportdesk.agents.router INFO routed to triage (The user cannot
access their account.): "can't log into my account, tried ..."
... supportdesk.services.desk_service INFO ticket: account_access / medium
Ticket created:
  customer: (not given)
  category: account_access
  urgency: medium
  summary: Customer can't log in even after resetting the password twice.
You: /quit
```

(Log lines are shortened with `...` and wrapped to fit the page.)

Now try an unclear one, like `"just wondering what your refund policy is, no rush"`, and read the log's reason. Whichever agent it picks, you can see *why* — that's what the checklist's "logged, with the reason" means.

Difference from Basic: the router understands wording, not just keywords, and every decision is logged with a reason. A failed routing call falls back to the safe agent. The flow sits in a service, so `main.py` stays thin. Step 3's error handling is kept. And the routing has its own test cases.

Which one should you actually write? Intermediate — it meets every checklist item. The keyword router is still worth building first, for 10 minutes: it's free, and it shows you exactly which messages a word list gets wrong — which is why the model-call router is worth one extra small call per message.